CS F363 Context-Free Grammar for the Toy Language

Hardik Agrawal - 2022A7PS0045H (f20220045@hyderabad.bits-pilani.ac.in)
Kalash Bhattad - 2022A7PS0065H (f20220065@hyderabad.bits-pilani.ac.in)
RVS Aashrey Kumar - 2022A7PS0160H (f20220160@hyderabad.bits-pilani.ac.in)
Pratyush Bindal - 2022A7PS0119H (f20220119@hyderabad.bits-pilani.ac.in)

<alpha> -> a | b | c |....| z
<digit> -> 0 | 1 |....| 9
<nonzerodigit> -> 1 | 2 |.....|9
<digit_opt> -> <digits> | ε
<digits> -> <digit> | <digit><digit_opt>
<alphanum> -> <alpha> | <digits>
<octdigit> -> 0 | 1 |....| 7
<nonzerooctdigit> -> 1 | 2 | 3 | …… | 7
<oct_opt> -> <octdigits> | ε
<octdigits> -> <octdigit> | <octdigit><oct_opt>
<bindigit> -> 0 | 1
<nonzerobindigit> -> 1
<bin_opt> -> <bindigits> | ε
<bindigits> -> <bindigit> | <bindigit><bin_opt>
<specsymbols> -> + | - | % | / | * | < | > | = | _ | (|) | ; | , | : | { | }
<base> -> 2 | 8 | 10
<binnum> -> 0<bin_opt><nonzerodigit> | <nonzerodigit><bin_opt>
<octnum> -> 0<oct_opt><nonzerooctdigit> | <nonzerooctdigit><oct_opt>
<decnum> -> 0<digit_opt><nonzerodigit> | <nonzerodigit><digit_opt>
<binintegerconst> -> (<binnum>, 2)
<octintegerconst> -> (<octnum>, 8)
<deciintegerconst> -> (<decnum>, 10)
<integerconst> -> <binintegerconst> | <octintegerconst> | <deciintegerconst>
<charconst> -> ‘<char>’ | ‘<escchar>’
<char> -> <alpha> | <digits> | <specsymbols>
<escchar> -> \n | \t
<string> -> “<charseq>”
<charseq> -> <char><charseq> | ε
<keywords> -> int | char | if | else | while | for | main | begin | end | print | scan | program | VarDecl | inc | dec
<datatype> -> int | char
<identifier> -> <alpha><rest>

<rest> -> <restalphanum> | <restunderscore> | ε
<restalphanum> -> <alphanum><rest>
<restunderscore> -> _<restalphanum>
<vardeclblock> -> begin VarDecl: <vardecllist> end VarDecl
<vardecllist> -> <vardecl><vardecllist> | ε
<vardecl> -> (<identifier>, <datatype>); | (<identifier>[<posint>], <datatype>);
<posint> -> <posdigit><posint> | <posdigit>
<posdigit> -> 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
<arithoperator> -> + | - | * | / | %
<relop> -> = | > | < | >= | <= | <>
<assignop> -> := | += | -= | *= | *= | /= | %=
<separator> -> (|) | , | ; | { | } | “ | @
<arithresult> -> <arithexpr>=<result>
<result> -> <integerconst>
<arithexpr> -> <arithexpr> + <arithterm> | <arithexpr> - <arithterm> | <arithterm>
<arithterm> -> <arithterm> * <arithfactor> | <arithterm> / <arithfactor> | <arithterm> %
<arithfactor> | <arithfactor>
<arithfactor> -> <identifier> | <integerconst> | (<arithexpr>)
<assignexpr> -> <integerconst><assignop><integerconst> |
<identifier><arithoperator><identifier> | <identifier><arithoperator><integerconst> |
<integerconst><arithoperator><identifier>
<relexpr> -> <relexpr> = <relterm> | <relexpr> <> <relterm> | <relterm>
<relterm> -> <relterm> > <relfactor> | <relterm> < <relfactor> | <relterm> >= <relfactor> |
<relterm> <= <relfactor> | <relfactor>
<relfactor> -> <arithexpr> | (<relexpr>)
<blockstmt> -> begin<blockstmtlist>end
<blockstmtlist> -> <blockstmtinside><blockstmtlist> | <blockstmtinside>
<stmtlist> -> <stmt><stmtlist> | <stmt>
<stmt> -> <assignstmt> | <printstmt> | <scanstmt> | <condifstmt> | <condifelsestmt> |
<whilestmt> | <forstmt> | <blockstmt>
<blockstmtinside> -> <assignstmt> | <printstmt> | <scanstmt>
<printstmt> -> print(<string>); | print(<formattext>, <valuelist>);
<scanstmt> -> scan(<formatscan>, <varlist>);
<formattext> -> “<charseq@>”
<charseq@> -> <charseq>@<charseq@> | <charseq>
<formatscan> -> “<atsymbols>”
<atsymbols> -> @ | @, <atsymbols>
<valuelist> -> <value> | <value>,<valuelist>
<varlist> -> <identifier> | <identifier>,<varlist>
<value> -> <integerconst> | <charconst> | <identifier>
<assignstmt> -> <identifier><assignop><expr>;
<expr> -> <value> | <arithexpr>
<value> -> <integerconst> | <charconst> | <identifier>
<cond> -> <identifier> | <relexpr>

<condifstmt> -> if (<cond>) then <blockstmt>; | if <cond> then <blockstmt> | ε
<condifelsestmt> -> if (<cond>) then <blockstmt> else <blockstmt>; | if <cond> then <blockstmt> else <blockstmt> | ε
<loopstmt> -> <whilestmt> | <forstmt>
<whilestmt> -> while (<cond>) <blockstmt>; | while <cond> <blockstmt>; | ε
<forstmt> -> for <identifier> := <forval> to <forval> <step> do <blockstmt>;
<step> -> inc<forval> | dec<forval>
<forval> -> <integerconst> | <arithexpr>
<comment> -> // <charseq> \n | /* <charseq_with_new_line> */
<charseq_with_new_line> -> <char_with_new_line><charseq_with_new_line> | ε
<char_with_new_line> -> <alpha> | <digits> | <specsymbols> | \n
<program> -> begin program: <vardeclblock> <stmtlist> end program